A Non-Contact Doppler Radar System for Vital Sign and Motion Detection

Project Exhibition - I

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted to

VIT BHOPAL UNIVERSITY (M.P.)

Submitted by

HEMANT RAJ (25BEC10093)

HARSH (25BEC10054)

Under the Supervision of

DR. RUPESH KUMARI

Assistant Professor

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

VIT BHOPAL UNIVERSITY

BHOPAL (M.P.) - 466114

October - 2026

VIT BHOPAL UNIVERSITY BHOPAL (M.P.) 466114

SCHOOL OF ELECTRICAL & ELECTRONICS ENGG.

CANDIDATE'S DECLARATION

We hereby declare that the project report entitled "A Non-Contact Doppler Radar System for Vital Sign and Motion Detection" is our own work conducted under the supervision of Dr. Rupesh Kumari, Assistant Professor, School of Electrical & Electronics Engineering at VIT University, Bhopal.

We further declare that to the best of our knowledge this report does not contain any part of work that has been submitted for the award of any degree either in this university or in any other university / Deemed University without proper citation.

Hemant Raj (Registration No. 25BEC10093)

Harsh (Registration No. 25BEC10054)

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Date: _____

Dr. Rupesh Kumari

Assistant Professor, VIT Bhopal University

Digital Signature of Guide

VIT UNIVERSITY BHOPAL (M.P.) - 466114

SCHOOL OF ELECTRICAL & ELECTRONICS ENGG.

CERTIFICATE

This is to certify that the work embodied in this Project Exhibition - I report entitled "A Non-Contact Doppler Radar System for Vital Sign and Motion Detection" has been satisfactorily completed by Hemant Raj (Registration No. 25BEC10093) and Harsh (Registration No. 25BEC10054) in the School of Electrical & Electronics Engineering at VIT University, Bhopal. This work is a bonafide piece of work, carried out under my guidance in the School of Electrical & Electronics Engineering for the partial fulfilment of the degree of Bachelor of Technology.

Dr. Rupesh Kumari

Assistant Professor, VIT Bhopal University

Forwarded by

Dr. Sivasankaran V

Program Chair (Core ECE)

Approved by

Dr. M. Suresh

Dean, School of Electrical & Electronics Engineering

Acknowledgement

We would like to express our sincere thanks to our guide, Dr. Rupesh Kumari, for the constant support and guidance given throughout this project. We also thank the School of Electrical & Electronics Engineering and VIT Bhopal University for providing the facilities required to complete this work. Finally, we thank our friends and family for their help and encouragement.

Executive Summary

Breathing rate is an important vital sign, but most methods of measuring it need sensors to be worn on the body, which can be uncomfortable. This project presents a non-contact method to monitor human breathing using an HB100 Doppler radar sensor. The radar transmits a 10.525 GHz microwave signal. When the chest moves during breathing, the frequency of the reflected wave changes slightly (Doppler effect). The reflected signal is extremely weak (microvolts), so a two-stage analog circuit using LM358 operational amplifiers, powered from 3.3 V, was designed to amplify it 10,000 times and to filter out unwanted frequencies. The signal is then converted to digital form by the ADC of an ESP32 microcontroller, and the Fast Fourier Transform (FFT) is used to find its frequency content. The ESP32 calculates detection scores and a confidence level from the FFT, and the breathing rate is obtained from the strongest peak (a normal rate of 18 breaths per minute corresponds to 0.3 Hz). The results are shown on a web dashboard.

List of Figures

Figure No.	Caption / Title	Page No.
3.1	Block diagram of the breathing detection system	3
3.2	Circuit diagram of the two-stage amplifier with band pass filter	4
3.3	Hardware setup of the system on breadboards	6
4.1	Serial monitor output showing the detection scores	8
4.2	Web dashboard output	8
4.3	Part of the ESP32 program (comms_ui.cpp) in VS Code	9

List of Tables

Table No.	Caption / Title	Page No.
3.1	Detection parameters used in the program	6

List of Symbols & Abbreviations

AC	Alternating Current
ADC	Analog to Digital Converter
BPM	Breaths Per Minute
DC	Direct Current
FFT	Fast Fourier Transform
GHz	Gigahertz
HB100	Doppler Radar Sensor Module
Hz	Hertz
IF	Intermediate Frequency
LM358	Dual Operational Amplifier IC
OLED	Organic Light Emitting Diode
c	Speed of light (m/s)
f_d	Doppler frequency shift (Hz)
f_t	Transmitted frequency (Hz)
mV	Millivolt
v	Velocity of chest movement (m/s)
µF	Microfarad
µV	Microvolt
Ω	Ohm

Table of Contents

Front Page ..... **i**
Candidate's Declaration ..... **ii**
Certificate ..... **iii**
Acknowledgement ..... **iv**
Executive Summary ..... **v**
List of Figures ..... **vi**
List of Tables ..... **vi**
List of Symbols & Abbreviations ..... **vii**

Contents **Page No.**

1 INTRODUCTION **1**
1.1 Motivation 1
1.2 General Theory 1
1.3 Objectives 1
1.4 Brief Methodology 1
2 LITERATURE SURVEY **2**
2.1 Review of Previous Studies 2
2.2 Gap in Literature 2
3 PROBLEM FORMULATION AND PROPOSED METHODOLOGY **3**
3.1 Block Diagram 3
3.2 Doppler Effect Calculations 3
3.3 Analog Signal Conditioning 4
3.4 Digital Processing 5
3.5 Hardware Implementation 6
4 RESULTS AND DISCUSSION **7**
4.1 Serial Monitor Output 7
4.2 Web Dashboard 8
4.3 Code Reference 8
5 CONCLUSIONS AND FUTURE SCOPE **10**
5.1 Conclusions 10
5.2 Future Scope 10

REFERENCES **11**
APPENDICES **12**

CHAPTER 1

INTRODUCTION

1.1 Motivation

Breathing rate is one of the most important vital signs of the human body. Most devices used to measure it must be worn or attached to the chest. These devices can be uncomfortable, especially for babies, elderly people and for long-term monitoring. Therefore, a system that can measure breathing without touching the person is needed.

1.2 General Theory

The Doppler effect is the change in the frequency of a wave when the object reflecting it is moving. The HB100 module [4] transmits a continuous microwave signal of 10.525 GHz (X-band). When this wave is reflected by a moving object such as the human chest, the frequency of the reflected wave changes slightly. The difference between the transmitted and the received frequency is called the Doppler frequency shift (f_d). It appears at the output of the module as a very weak Intermediate Frequency (IF) signal, which carries the information about the chest movement.

1.3 Objectives

- To design a non-contact system that detects human breathing using an HB100 radar.
- To amplify a very weak microvolt signal to a readable voltage level.
- To filter out noise and isolate the breathing signal (normal rate about 0.3 Hz).
- To process the signal using FFT on an ESP32 microcontroller.
- To display the result on an OLED screen and a web dashboard.

1.4 Brief Methodology

1. Transmit and receive: the HB100 radar sends microwaves and receives the reflected signal.
2. Analog conditioning: the weak signal is amplified 10,000 times and filtered using LM358 op-amps.

3. Digitization: the 12-bit ADC of the ESP32 converts the analog voltage to digital numbers.
4. Digital processing: the FFT algorithm finds the frequency content of the signal.
5. Output: the result is displayed on the OLED screen and the web dashboard.

CHAPTER 2

LITERATURE SURVEY

2.1 Review of Previous Studies

[1] M. Celik (2022) has discussed an Arduino-based Doppler radar built with the HB100 module. In that project the Doppler signal is amplified by an op-amp circuit, converted into the speed of moving vehicles, shown on a 16x2 LCD and sent to the ThingSpeak website using an ESP8266 Wi-Fi module. The work shows that the HB100 can be used with low-cost boards, but it is used for vehicle speed and not for breathing.

[2] A. D. Droitcour et al. (2004) have developed Doppler radar chips in silicon technology (1.6 GHz and 2.4 GHz) that detected the movement due to heartbeat and respiration from a distance of 50 cm. They showed that range correlation reduces the residual phase noise, and that a quadrature (I/Q) receiver avoids the weak-signal (null) points.

[3] C. Li et al. (2013) have reviewed the recent advances in Doppler radar sensors for noncontact healthcare monitoring.

2.2 Gap in Literature

Many existing vital sign systems are expensive and use complex RF components, while the low-cost HB100 projects available mostly measure the speed of vehicles [1]. There is a need for a low-cost and portable system that uses easily available components such as the HB100 and ESP32 to monitor breathing, with signal conditioning and FFT done locally. This justifies the present project.

CHAPTER 3

PROBLEM FORMULATION AND PROPOSED METHODOLOGY

3.1 Block Diagram

The working of the system is shown in Fig. 3.1. The movement of the chest during breathing is detected by the HB100 radar. The weak output of the radar is amplified and filtered by a two-stage LM358 circuit. The ESP32 converts the signal to digital form, finds the frequency content using FFT and shows the result on the display.

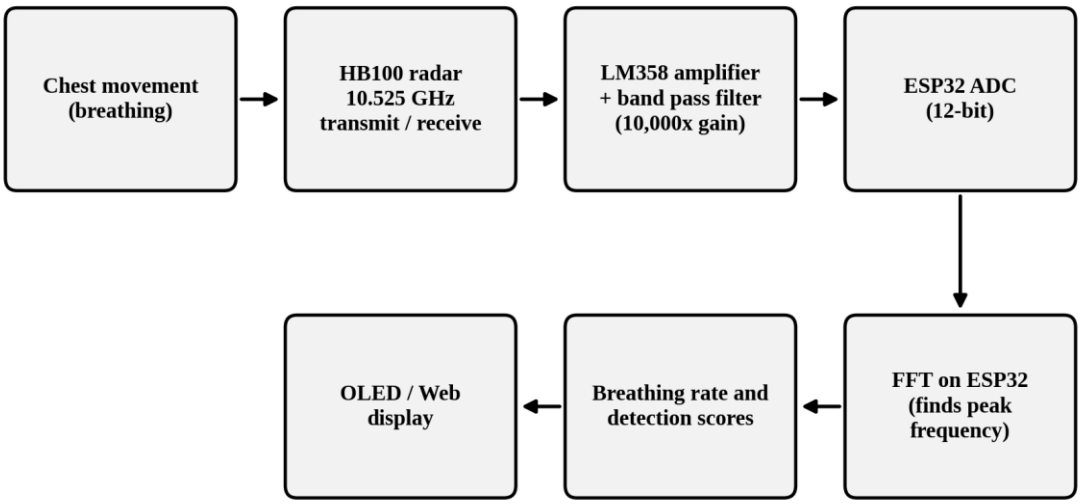


Fig. 3.1 Block diagram of the breathing detection system

3.2 Doppler Effect Calculations

The HB100 transmits at a frequency f_t = 10.525 GHz. If the chest moves with a velocity v, the Doppler frequency shift f_d is given by Eq. (3.1), where c = 3×10^8 m/s is the speed of light.

$$f_d = \frac{2vf_t}{c} \tag{3.1}$$

Putting the values of f_t and c in Eq. (3.1) gives Eq. (3.2), where v is in m/s. This is the same as the 19.49 Hz per km/h given in the datasheet [4].

$$f_d = 70.2 \times v \tag{3.2}$$

For normal breathing, the frequency is about 0.3 Hz (18 breaths per minute). So v = 0.3 / 70.2 ≈ 0.004 m/s, that is, about 4 mm/s. This means that the chest moves at about 4 mm per second during breathing, which gives a Doppler shift of 0.3 Hz.

3.3 Analog Signal Conditioning

The raw signal from the radar is only about 10 µV and it rides on a DC voltage. It must be amplified 10,000 times and all frequencies outside the breathing range (about 0.1 Hz to 3 Hz) must be removed. The circuit used is shown in Fig. 3.2. The gain is divided into two stages to avoid noise amplification and instability. Both LM358 op-amps [5] run from a single 3.3 V supply.

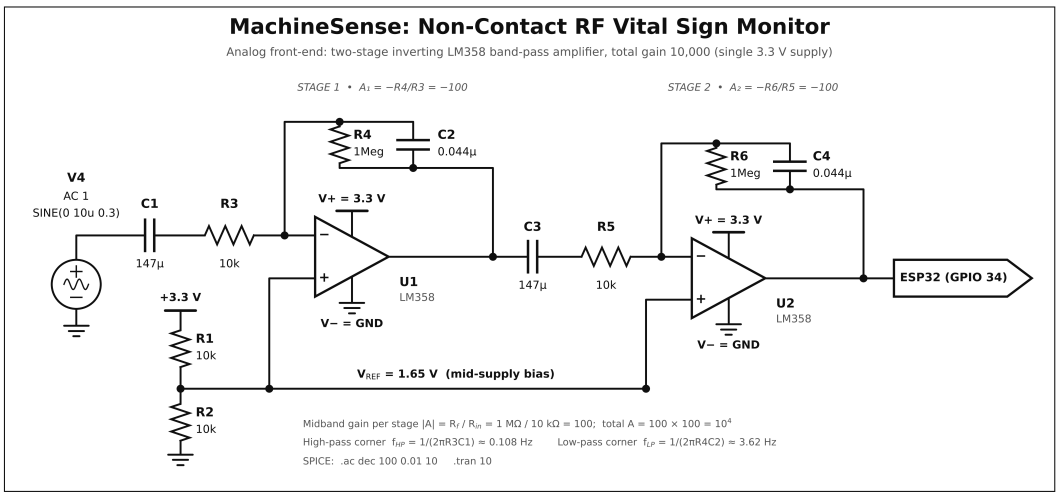


Fig. 3.2 Circuit diagram of the two-stage amplifier with band pass filter (LTspice; V4 is the test source used in simulation)

3.3.1 Need for 10,000 times gain

- A 10 µV signal becomes 100 mV, which rises above the random noise.
- 100 mV is large enough to be read by the ADC of the ESP32.
- The output is biased at 1.65 V and swings only about 100 mV, so clipping is avoided.
- DC drift is blocked by AC coupling at the input of each stage.

3.3.2 Stage 1 amplifier (U1)

Capacitor C1 (147 µF) blocks the DC offset and allows only the AC radar signal to pass. C1 and resistor R3 (10 kΩ) form a high pass filter, and C2 (0.044 µF) in parallel with R4 (1 MΩ) forms a low pass filter. The corner frequency of each filter is given by Eq. (3.3). It is about 0.108 Hz for C1 and R3, and about 3.6 Hz for C2 and R4.

$$f_c = \frac{1}{2\pi RC} \tag{3.3}$$

The gain of the stage is given by Eq. (3.4), where R_f is the feedback resistor and R_{in} is the input resistor. Here R_f = R4 = 1 MΩ and R_{in} = R3 = 10 kΩ, so the gain is 100.

$$A = \frac{R_f}{R_{in}} \tag{3.4}$$

3.3.3 Stage 2 amplifier (U2)

Capacitor C3 (147 µF) blocks any new DC level. The gain is R6 / R5 = 1 MΩ / 10 kΩ = 100, and C4 (0.044 µF) in parallel with R6 gives the same low pass action as in stage 1. The total gain of the two stages is given by Eq. (3.5).

$$A_{total} = A_1 \times A_2 = 100 \times 100 = 10{,}000 \tag{3.5}$$

3.3.4 Band pass response

Each stage has a high pass corner at about 0.1 Hz and a low pass corner at about 3.6 Hz, so together they form a band pass filter that passes roughly 0.1 Hz to 3.6 Hz. DC (0 Hz) and high-frequency noise are blocked, while the breathing signal (about 0.3 Hz) passes.

3.3.5 Virtual ground

Two resistors R1 and R2 (10 kΩ each) form a voltage divider that gives a reference of 1.65 V (half of the 3.3 V supply). This voltage is applied to the non-inverting (+) input of both op-amps. If the op-amps were biased at 0 V, the negative half of the AC wave would be cut off. With the 1.65 V reference, the AC wave can swing safely above and below this mid-point.

3.4 Digital Processing

3.4.1 ADC

The output of stage 2 is connected directly to the ADC input of the ESP32. Since the op-amps run from 3.3 V, the output cannot go above the 3.3 V limit of the ADC, so no

voltage divider is needed. The signal is 1.65 V DC plus about 100 mV AC. The 12-bit ADC divides the range 0 V to 3.3 V into 4096 steps. The size of one step is given by Eq. (3.6).

$$V_{step} = \frac{3.3\ V}{4096} \approx 0.8\ mV \tag{3.6}$$

3.4.2 FFT and detection

The ESP32 stores the ADC readings in a buffer. This list of readings is the time-domain signal (voltage changing with time). The FFT algorithm converts it into the frequency domain, which gives the strength (magnitude) of each frequency bin. The program then calculates two scores in percent: score A compares the FFT magnitudes with a calibrated baseline (noise floor), and score C measures how much the spectrum has changed since the previous FFT window. They are combined into a fused score F given by Eq. (3.7). The parameters used are listed in Table 3.1.

$$F = 0.60A + 0.40C \tag{3.7}$$

A confidence level (CONF) from 0 to 5 shows how steadily a person is detected. The breathing rate is obtained from the frequency of the strongest peak using Eq. (3.8). For example, a peak at 0.3 Hz means 0.3 × 60 = 18 breaths per minute (BPM).

$$BPM = f_{peak} \times 60 \tag{3.8}$$

Table 3.1 Detection parameters used in the program

Parameter	Value	Meaning
FUSION_THRESHOLD	0.60	Detection threshold on fused score F
WEIGHT_A	0.60	Weight of score A in F
WEIGHT_C	0.40	Weight of score C in F
MIN_ABS_MAG	150	Minimum absolute FFT magnitude

3.5 Hardware Implementation

The circuit was built on breadboards using the HB100 module, the two LM358 stages and an ESP32 board, as shown in Fig. 3.3.

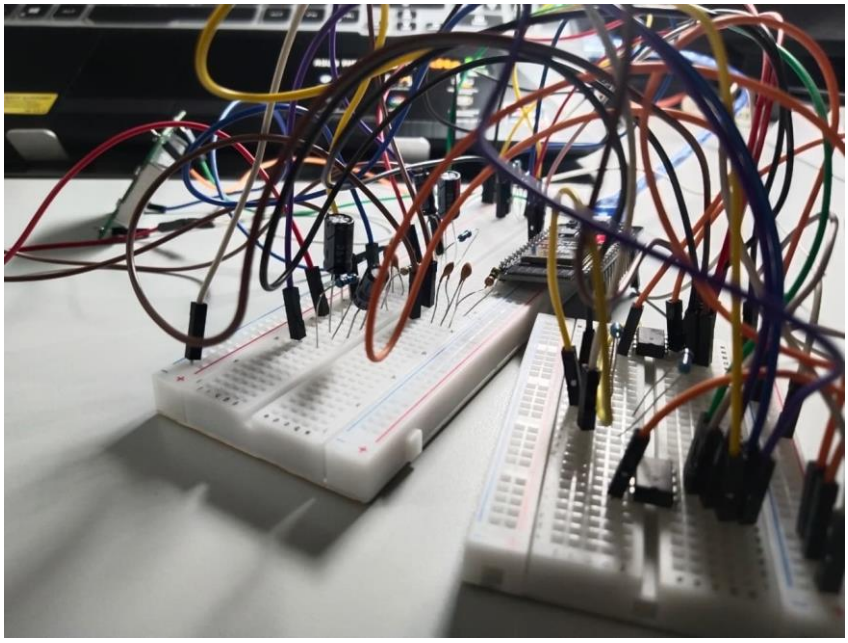


Fig. 3.3 Hardware setup of the system on breadboards

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Serial Monitor Output

The program prints the result of every FFT cycle on the serial monitor, as shown in Fig. 4.1. Each line gives the cycle number (#), the scores A, B and C and the fused score F in percent, the confidence level CONF (0 to 5) and the FFT magnitudes of the low-frequency bins (BINS). Score B is an extra score from an earlier method used during testing and is not part of the fused score. The BINS values change from cycle to cycle, which shows that the program follows the chest movement continuously, and CONF stays at 4/5 or 5/5 in all the cycles shown.

```
[MachineSense] #118  A=72%  B=59%  C=45%  F=61%  CONF=5/5  BINS=[294,738,679,272,161]
[MachineSense] #119  A=62%  B=35%  C=33%  F=45%  CONF=5/5  BINS=[307,522,426,343,401]
[MachineSense] #120  A=69%  B=33%  C=39%  F=49%  CONF=5/5  BINS=[381,354,562,554,343]
[MachineSense] #121  A=36%  B=9%  C=37%  F=27%  CONF=5/5  BINS=[112,344,252,178,213]
[MachineSense] #122  A=0%  B=0%  C=76%  F=19%  CONF=5/5  BINS=[98,21,15,28,59]
[MachineSense] #123  A=46%  B=31%  C=34%  F=38%  CONF=4/5  BINS=[184,271,236,199,152]
[MachineSense] #124  A=47%  B=29%  C=33%  F=37%  CONF=5/5  BINS=[411,180,210,139,217]
[MachineSense] #125  A=5%  B=0%  C=50%  F=14%  CONF=5/5  BINS=[280,170,95,184,46]
[MachineSense] #126  A=44%  B=35%  C=41%  F=40%  CONF=4/5  BINS=[441,159,181,210,170]
[MachineSense] #127  A=51%  B=44%  C=47%  F=47%  CONF=5/5  BINS=[432,187,638,817,474]
[MachineSense] #128  A=76%  B=37%  C=37%  F=53%  CONF=5/5  BINS=[264,595,586,409,314]
[MachineSense] #129  A=47%  B=0%  C=49%  F=31%  CONF=5/5  BINS=[24,175,117,93,173]
[MachineSense] #130  A=68%  B=35%  C=36%  F=49%  CONF=5/5  BINS=[743,656,438,216,307]
[MachineSense] #131  A=80%  B=43%  C=46%  F=58%  CONF=5/5  BINS=[513,428,468,369,176]
[MachineSense] #132  A=67%  B=22%  C=48%  F=47%  CONF=5/5  BINS=[102,69,124,157,311]
[MachineSense] #133  A=47%  B=23%  C=43%  F=37%  CONF=5/5  BINS=[250,95,84,230,149]
```

Fig. 4.1 Serial monitor output showing the detection scores

4.2 Web Dashboard

The ESP32 sends the results to a web dashboard through a WebSocket connection. The dashboard shows the live output of the system, as given in Fig. 4.2.



Fig. 4.2 Web dashboard output

4.3 Code Reference

The program is written in C++ using PlatformIO in VS Code and is divided into the files main, radar_sensor, signal_processing, detection and comms_ui. The file comms_ui handles the WebSocket communication with the dashboard (Fig. 4.3). The

complete code is given in Appendix-I and is available on GitHub at https://github.com/harsh0028-boop

and https://github.com/Hemant-Raj-1101

Machinesense > src > comms_ui.cpp > ...

```
13  // Track connected clients (for debug logging only)
14  static uint8_t connectedClients = 0;
15
16  // ----------------------------------------------------------------------
17  // WebSocket Event Handler
18  // Called by the library whenever a client connects, disconnects,
19  // or sends data. Runs in the context of webSocket.loop().
20  // ----------------------------------------------------------------------
21  static void webSocketEvent(uint8_t clientNum, WStype_t type,
22                             uint8_t *payload, size_t length) {
23      switch (type) {
24
25          case WStype_DISCONNECTED:
26              if (connectedClients > 0) {
27                  connectedClients--;
28              }
29              break;
30
31          case WStype_CONNECTED: {
32              connectedClients++;
33
34              IPAddress ip = webSocket.remoteIP(clientNum);
35
36              // Send a welcome/handshake string so the dashboard knows
37              // it is talking to the correct MachineSense device
38              webSocket.sendTXT(
39                  clientNum,
40                  "$MachineSense,STATUS,CONNECTED"
41              );
42
```

Fig. 4.3 Part of the ESP32 program (comms_ui.cpp) in VS Code

CHAPTER 5

CONCLUSIONS AND FUTURE SCOPE

5.1 Conclusions

- The HB100 radar was able to detect the chest movement without any contact with the person.
- The two-stage LM358 circuit, powered from 3.3 V, amplified the weak microvolt signal 10,000 times and its band pass filter (about 0.1 Hz to 3.6 Hz) removed DC and high-frequency noise.
- The ESP32 digitized the signal, ran the FFT and calculated the detection scores and confidence level in real time.
- The results were shown on a web dashboard, and the system was built using low-cost and easily available components.

5.2 Future Scope

- Heart rate detection: the filter can be adjusted to also capture the higher-frequency heartbeat signal.
- Wireless transmission: the web server can be extended with a Bluetooth link to a mobile app.
- Miniaturization: a custom PCB can be designed to make the device small and portable.

REFERENCES

[1] M. Celik, “ArduinoHB100Radar: HB100 radar speed measurement using ESP8266 and ThingSpeak,” GitHub repository, 2022. [Online]. Available: https://github.com/celikslab/ArduinoHB100Radar (accessed Oct. 8, 2026).

[2] A. D. Droitcour, O. Boric-Lubecke, V. M. Lubecke, J. Lin, and G. T. A. Kovacs, “Range correlation and I/Q performance benefits in single-chip silicon Doppler radars for noncontact cardiopulmonary monitoring,” IEEE Trans. Microw. Theory Techn., vol. 52, no. 3, pp. 838–848, Mar. 2004.

[3] C. Li, V. M. Lubecke, O. Boric-Lubecke, and J. Lin, “A review on recent advances in Doppler radar sensors for noncontact healthcare monitoring,” IEEE Trans. Microw. Theory Techn., vol. 61, no. 5, pp. 2046–2060, May 2013.

[4] “HB100 Microwave Motion Sensor,” Agilent Technologies, Datasheet.

[5] “LM358 Low-Power Dual Operational Amplifier,” Texas Instruments, Datasheet.

APPENDIX-I ESP32 Source Code

```
#include "globals.h"
#include "radar_sensor.h"
#include "signal_processing.h"
#include "detection.h"
#include "comms_ui.h"


// ================================================================================
// MachineSense - Global RTOS Handles
// ================================================================================
QueueHandle_t rawDataQueue;              // Raw radar samples
QueueHandle_t processedDataQueue;        // Processed vital-sign data
QueueHandle_t dashboardQueue;            // Dashboard telemetry
SemaphoreHandle_t detectionSemaphore;    // Human detection trigger


// ================================================================================
// MachineSense - Global System State
// ================================================================================
volatile SystemState systemState = STATE_CALIBRATING;


// ================================================================================
// SETUP
// ================================================================================
void setup()
{
  Serial.begin(115200);              // Init Serial
  delay(1000);                       // Wait for Serial stability


  // ------------------------------------------------------------------
  // Boot Information
  // ------------------------------------------------------------------
  Serial.println();
  Serial.println("================================================");
  Serial.println("      MachineSense System Boot");
  Serial.println("================================================");
  Serial.print("VitalSignData size: ");
  Serial.println(sizeof(VitalSignData));
  Serial.print("DashboardTelemetry size: ");
  Serial.println(sizeof(DashboardTelemetry));
  Serial.print("Free heap before queues: ");
  Serial.println(ESP.getFreeHeap());


  // ================================================================
  // 1. CREATE FREERTOS QUEUES AND SEMAPHORES
  // ================================================================
  rawDataQueue = xQueueCreate(2048, sizeof(uint16_t));       // Raw radar samples
  processedDataQueue = xQueueCreate(4, sizeof(VitalSignData));// Vital-sign data
  dashboardQueue = xQueueCreate(4, sizeof(DashboardTelemetry));// Telemetry
```

```
detectionSemaphore = xSemaphoreCreateBinary();              // Detection flag


Serial.print("Free heap after queues: ");
Serial.println(ESP.getFreeHeap());


// ============================================================================
// 2. VERIFY RTOS OBJECT CREATION
// ============================================================================
if (rawDataQueue == NULL ||
    processedDataQueue == NULL ||
    dashboardQueue == NULL ||
    detectionSemaphore == NULL)
{
    Serial.println();
    Serial.println("================================================");
    Serial.println("ERROR: MachineSense RTOS object creation failed!");
    Serial.println("System halted.");
    Serial.println("================================================");
    while (1) { delay(1000); }          // Halt on failure
}


Serial.println();
Serial.println("MachineSense RTOS Queues: OK");
Serial.println("MachineSense Detection Semaphore: OK");


// ============================================================================
// 3. INITIALIZE COMMUNICATIONS / WIFI
// ============================================================================
Serial.println();
Serial.println("Initializing MachineSense communications...");
initCommsWiFi();
Serial.println("MachineSense communications initialized.");


// ============================================================================
// 4. CREATE FREERTOS TASKS
//    Priority: 4=Radar, 3=Processing, 2=Detection, 1=Comms/UI
// ============================================================================
if (xTaskCreate(vRadarAcquisitionTask, "RadarTask", 2048, NULL, 4, NULL) != pdPASS) {
    Serial.println("ERROR: Failed to create RadarTask; system halted.");
    while (1) delay(1000);
}


if (xTaskCreate(vSignalProcessingTask, "ProcessTask", 8192, NULL, 3, NULL) != pdPASS) {
    Serial.println("ERROR: Failed to create ProcessTask; system halted.");
    while (1) delay(1000);
}
```

```
  if (xTaskCreate(vDetectionTask, "DetectTask", 6144, NULL, 2, NULL) != pdPASS) {
    Serial.println("ERROR: Failed to create DetectTask; system halted.");
    while (1) delay(1000);
  }

  if (xTaskCreate(vCommsUITask, "CommsTask", 8192, NULL, 1, NULL) != pdPASS) {
    Serial.println("ERROR: Failed to create CommsTask; system halted.");
    while (1) delay(1000);
  }

  // ============================================================================
  // SYSTEM READY
  // ============================================================================
  Serial.println();
  Serial.println("============================================");
  Serial.println("      MachineSense System Ready");
  Serial.println("============================================");
  Serial.println("Radar Task      : Priority 4");
  Serial.println("Processing Task  : Priority 3");
  Serial.println("Detection Task   : Priority 2");
  Serial.println("Comms/UI Task    : Priority 1");
  Serial.println("============================================");
  Serial.println();
}

// ============================================================================
// MAIN LOOP
// ============================================================================
void loop()
{
  vTaskDelete(NULL);
}
```